

10. Introducción JavaFX

Objetivos

- Describir la historia de la programación gráfica en Java;
- explicar la estructura y ciclo de vida de una aplicación **JavaFX**;
- generar formas gráficas 2D en **JavaFX**;
- construir aplicaciones gráficas interactivas en **JavaFX** usando componentes comunes tales como botones, campos de texto y etiquetas;
- programar un control **JavaFX** para escuchar eventos usando una **expresión lambda**;
- hacer uso de la variedad de contenedores **JavaFX** diferentes;
- crear bordes, fuentes y colores;
- dar formato a números decimales para que aparezcan en una forma apropiada en una aplicación gráfica.

10.1 Introducción

En este capítulo se revisan conceptos de programación gráfica. Se inicia el cambio de programas que usan la pantalla de texto a programas con interfaz gráfica usando ventanas.

Para lograr lo anterior se usará el paquete gráfico de Java conocido como **JavaFX**. Este paquete proporciona todas las herramientas gráficas y componentes que se requieren para producir el tipo de interfaces gráficas que se usan por todos en las aplicaciones modernas diarias.

10.2 Breve historia de Gráficos en Java

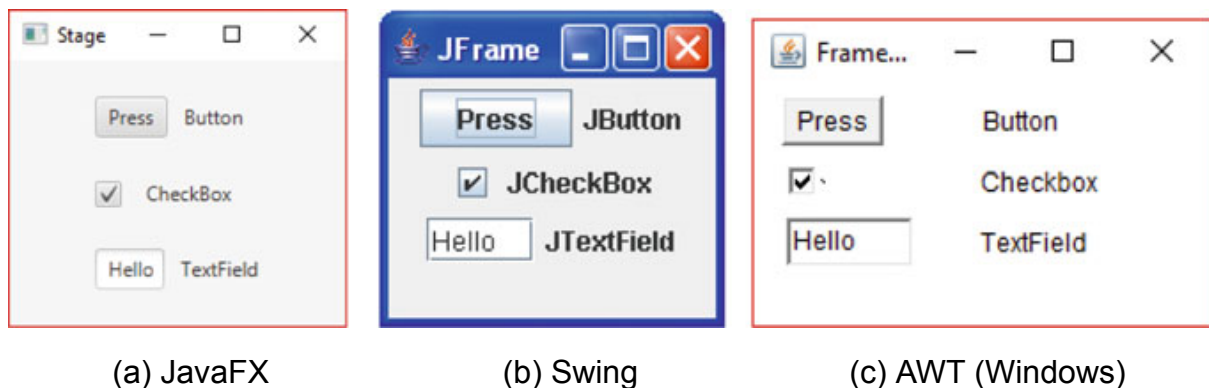
En las primeras versiones de Java, la programación gráfica se lograba exclusivamente haciendo uso del paquete **AWT (Abstract Window Toolkit)**. La idea con AWT fue proporcionar un sistema de gráficos en el cual cualquier componente creado está asociado con el componente correspondiente en el sistema operativo nativo. Si se creaba un componente gráfico, como un botón o campo de texto, el componente sería proporcionado por el sistema operativo, por ejemplo por Windows o por macOS, así el botón o campo de texto se vería exactamente como uno de los

usados en el sistema operativo particular. Componentes que dependen del sistema operativo nativo hacen uso extensivo de los recursos del sistema y son por lo tanto descritos como componentes de **peso pesado**.

Como AWT usaba muchos recursos, y por las diferencias funcionales entre sistemas operativos, no tuvo éxito completo, y este sistema fue reemplazado por el paquete **Swing**. Las clases Swing están escritas la mayor parte en Java, y como no dependen de los componentes del sistema estos son conocidos como componentes de **peso ligero**. A diferencia de AWT, los componentes Swing se ven igual sin importar el sistema operativo donde el programa se esté corriendo.

Swing fue la plataforma principal para producir gráficos Java entre los años 2000 y 2014. Sin embargo, su apariencia se volvió bastante anticuada en comparación con los gráficos actuales que se ejecutan en múltiples dispositivos, y con el lanzamiento de Java 8 en 2014 llegó la última versión de una nueva tecnología conocida como JavaFX, junto con el anuncio de que Swing no se desarrollará más, sin embargo se continuará empaquetando con Java.

En la siguiente figura se comparan algunos componentes gráficos comunes usando las tres tecnologías.



10.3 Descripción General de JavaFX

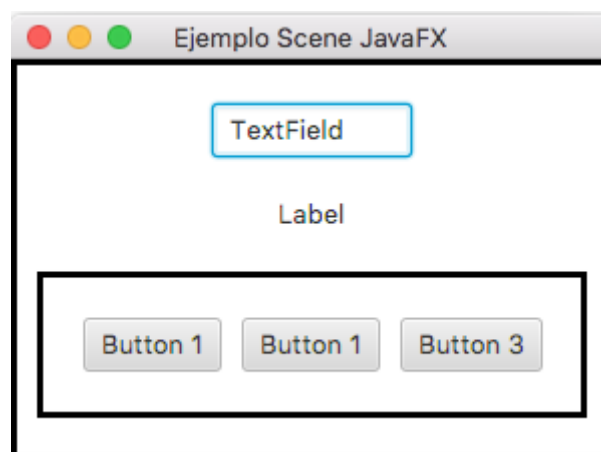
En el texto se usará solo JavaFX para las aplicaciones gráficas. Un programa JavaFX es referido como una **aplicación**. La clase JavaFX propia extenderá la clase Application, por lo cual se necesita la siguiente sentencia de importación:

```
import javafx.application.Application;
```

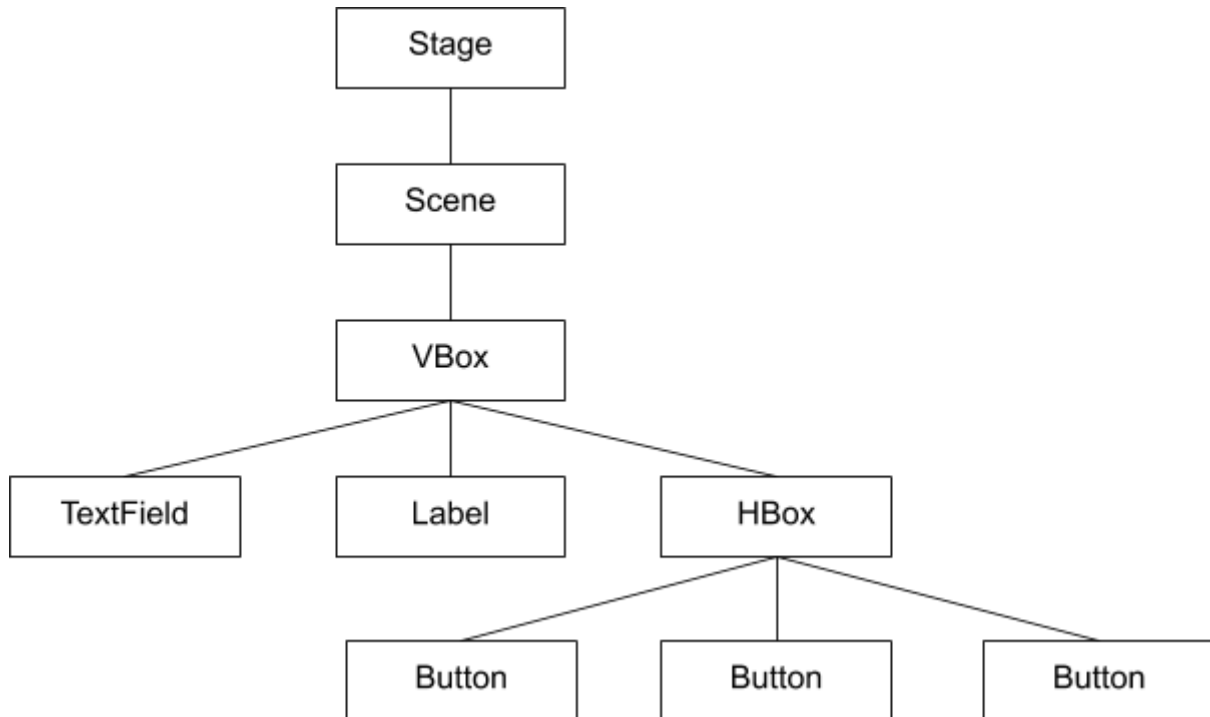
La ventana de nivel superior en la que se ejecuta la aplicación se denomina **Stage** (escenario), será una ventana como la primera que se muestra en la figura previa, pero si se ejecuta el programa en modo pantalla completa, entonces la pantalla se convierte en el escenario. Algunas aplicaciones pueden ejecutarse en un navegador, en cuyo caso el navegador es el escenario. El contenido del escenario,

el gráfico propio, se denomina **Scene** (escena) (interior del cuadro negro mas grande de la figura) y también es denominado como un **gráfico de escena**. Los elementos que componen la escena se denominan **nodos**. Con bastante frecuencia, son el tipo de componentes que permiten la interacción con el usuario, como botones, campos de texto, etiquetas y casillas de verificación, que son llamados colectivamente como **controles**. También pueden ser formas gráficas 2D o 3D. Pero los nodos también pueden ser **contenedores**. Los contenedores son componentes que contienen otros nodos, y cada contenedor organiza los nodos de una manera particular, por ejemplo, verticalmente, horizontalmente, en una cuadrícula o apilados uno encima del otro. Normalmente, no se vería el contenedor, pero es posible ponerle un borde si se quiere. Es importante destacar que los contenedores pueden contener otros contenedores, por lo que se puede desarrollar una jerarquía en la escena. Normalmente se coloca un solo nodo de nivel superior en la escena, y esto es conocido como el nodo **raíz**. Se usan los términos **padre** e **hijo** para los nodos que contienen y contenidos, respectivamente.

En la siguiente figura se tiene una escena simple en la cual el nodo raíz es un VBox, es es un contenedor que arregla sus nodos hijo verticalmente. Se le ha dado un borde negro para que se pueda ver. El VBox tiene tres hijos, un TextField, un Label y un HBox, al cual también se le ha puesto un borde. Un HBox es parecido a un VBox, pero arregla sus nodos hijos horizontalmente. En este caso tiene tres nodos hijos los cuales son Button.



Para ayudar a entender la forma como la escena es construida en una forma jerárquica en la figura siguiente se tiene la jerarquía de la escena gráfica previa.



Cuando una aplicación JavaFX inicia, hay tres métodos que son llamados en orden. Estos son:

```
void init()  
abstract void start(Stage stage)  
void stop()
```

El método `init()` es en dónde se podrían colocar rutinas que requieran ser ejecutadas antes de que la propia aplicación inicie, mientras el método `stop()` es dónde se colocaría el código que se quisiera ejecutar después de que la aplicación termine. En este capítulo no se considerarán estos dos métodos. Se revisará el método `start()`, el cual es un método **abstracto** y por lo tanto tiene que ser codificado. En este método se pone el código de la aplicación. Se puede dividir agregando algunos métodos auxiliares, pero es con este método que la propia aplicación inicia.

Si la aplicación JavaFX es ejecutada desde la línea de comandos no se requiere un método `main()` para lanzarla. Tampoco se necesita un método `main()` si esta es desplegada como un archivo `.java`, lo cual será revisado en el segundo semestre. Si se ejecuta el programa dentro de un IDE, entonces se requiere un método `main()`, y por esa razón se incluye ese método con cada aplicación desarrollada. El método `main()` que se revisará tiene la siguiente forma:

```
public static void main(String[] args) {
```

```
    launch(args);  
}
```

Como se observa el método `main()` llama al método `launch()` de la aplicación, y pasa a este los argumentos recibidos por el propio método `main()`.

El método `launch()` es un método **static**, y se puede usar para lanzar una aplicación JavaFX desde otro programa. Está sobrecargado para aceptar el nombre del archivo compilado `.class` como su primer parámetro. Si se quiere que un programa llamado `LanzarAplicacion` corra una aplicación llamada `MiApp`, se haría de esta forma:

```
import javafx.application.Application;  
  
class LanzarAplicacion {  
    public static void main(String[] args) {  
        Application.launch(MiApp.class, args);  
    }  
}
```

Una vez que se sabe cómo una aplicación JavaFX está estructurada, y se conoce la secuencia en la que se llaman sus métodos, se puede pasar a desarrollar la primera aplicación gráfica.

10.4 Gráficos 2D. Clase `CaraSonriente`

La primera aplicación gráfica creará una cara sonriente como se muestra en la figura siguiente:



Es una aplicación simple porque no hay interacción con el usuario, no obstante introduce muchos conceptos nuevos. En particular muestra cómo crear una escena, agregar elementos a la escena, y agregar la escena al escenario. También introduce los gráficos 2D, los cuales permiten dibujar formas de dos dimensiones tales como círculos, líneas, elipses, rectángulos y arcos. Se dibujan círculos para la cara y los ojos, y un arco para la boca. Se ve también cómo crear texto el cual se configura para usar colores y fuentes diferentes.

Como se explicó en la sección previa, se ha incluido un método `main()` el cual lanza la aplicación. El código completo es mostrado a continuación:

```
import javafx.application.Application;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.shape.Arc;
import javafx.scene.shape.ArcType;
import javafx.scene.shape.Circle;
import javafx.scene.text.Font;
import javafx.scene.text.Text;
import javafx.stage.Stage;

public class CaraSonriente extends Application
{
    @Override
    public void start(Stage stage) {
        // crear y configurar el círculo principal (cara)
        Circle cara = new Circle(125, 125, 80);
        cara.setFill(Color.YELLOW);
        cara.setStroke(Color.RED);

        // crear y configurar el círculo para ojo derecho
        Circle ojoDerecho = new Circle(86, 100, 10);
        ojoDerecho.setFill(Color.YELLOW);
        ojoDerecho.setStroke(Color.BLUE);

        // crear y configurar el círculo para ojo izquierdo
        Circle ojoIzquierdo = new Circle(162, 100, 10);
        ojoIzquierdo.setFill(Color.YELLOW);
        ojoIzquierdo.setStroke(Color.BLUE);

        // crear y configurar una boca sonriente
        Arc boca = new Arc(125, 150, 45, 35, 0, -180);
        boca.setFill(Color.YELLOW);
        boca.setStroke(Color.BLUE);
        boca.setType(ArcType.OPEN);
    }
}
```

```

        // crear y configurar el texto
        Text leyenda = new Text(70, 240, "Cara Sonriente");
        leyenda.setFill(Color.BLUE);
        leyenda.setFont(Font.font("Verdana", 15));

        // crear un grupo que tenga todos los componentes
        Group raiz = new Group(
            cara, ojoDerecho, ojoIzquierdo, boca, leyenda);

        // crear y configurar una escena nueva
        Scene escena = new Scene(raiz, 250, 275, Color.YELLOW);

        // agregar la escena al escenario y poner título
        stage.setScene(escena);
        stage.setTitle("Cara Sonriente");

        // mostrar el escenario
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

Las cláusulas **import** muestran que todas las clases vienen de un paquete llamado `javafx`, dónde también se ve que tiene muchos subpaquetes incluyendo `scene` y `stage`.

```

import javafx.application.Application;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.shape.Arc;
import javafx.scene.shape.ArcType;
import javafx.scene.shape.Circle;
import javafx.scene.text.Font;
import javafx.scene.text.Text;
import javafx.stage.Stage;

```

Todos los programas JavaFX corren como una aplicación, y por lo tanto se tiene que extender la clase `Application`, como se observa en la cabecera de la clase:

```

public class CaraSonriente extends Application {

```

Una `Application` requiere que se codifique el método `start()`, del cual se comentó en la sección previa, por lo que la cabecera es:

```
public void start(Stage stage) {
```

Cuando `start()` es llamado, automáticamente se le envía un objeto de la clase `Stage`, el cual será el contenedor principal del gráfico.

La primera sentencia que es ejecutada dentro del método `start()` es para crear y configurar el círculo principal de la cara:

```
Circle cara = new Circle(125, 125, 80);  
cara.setFill(Color.YELLOW);  
cara.setStroke(Color.RED);
```

La clase `Circle`, del paquete `javafx.scene.shape`, tiene varios constructores. El constructor usado toma tres parámetros de tipo **double**. Los primeros dos parámetros representan la posición *x* e *y* respectivamente del centro del círculo, con respecto a la esquina superior izquierda del nodo padre, medido en píxeles. El tercer parámetro representa el radio del círculo, también en píxeles. Se verá más adelante que se escogió que la escena inicial sea de 250 x 275 píxeles, así el círculo con centro en (125,125) estará centrado horizontalmente, pero dejará espacio suficiente para una leyenda.

Se han usado otros dos métodos de `Circle`, `setFill()` y `setStroke()`, para poner el color de relleno y el color de la línea del círculo. Para cada uno de estos se ha pasado un atributo predefinido de la clase `Color`, la cual reside en `javafx.scene.paint`. En la sección [10.10](#) se revisa cómo crear colores propios, pero la biblioteca `paint` proporciona una gran cantidad de colores.

De forma similar se dibuja el ojo derecho e izquierdo:

```
Circle ojoDerecho = new Circle(86, 100, 10);  
ojoDerecho.setFill(Color.YELLOW);  
ojoDerecho.setStroke(Color.BLUE);  
  
Circle ojoIzquierdo = new Circle(162, 100, 10);  
ojoIzquierdo.setFill(Color.YELLOW);  
ojoIzquierdo.setStroke(Color.BLUE);
```

Para decidir la posición exacta para dibujar los círculos se puede hacer una estimación y revisar cómo se ve, luego cambiar los valores hasta que se logre el resultado deseado.

Para la boca sonriente se emplea el código siguiente:

```
Arc boca = new Arc(125, 150, 45, 35, 0, -180);  
boca.setFill(Color.YELLOW);  
boca.setStroke(Color.BLUE);  
boca.setType(ArcType.OPEN);
```

Crear un objeto de la clase `Arc` dibuja una parte, o todo, de una elipse. El constructor usado toma seis parámetros. Los primeros dos representan la posición del centro de la elipse. Los siguientes dos son el radio horizontal y el vertical respectivamente. El quinto representa el ángulo a partir del cual se inicia el dibujo del arco. El último representa el tamaño del ángulo a través del cual el arco es dibujado.

Para este caso se escogieron radios para dar un arco de elipse que sea más ancho que alto. Se escogió un ángulo de inicio de 0° y el valor del ángulo final es puesto a -180° . El signo negativo indica que este ángulo es formado moviéndose desde el ángulo inicial en dirección de las manecillas del reloj. Con un valor positivo se indica una dirección contraria a las manecillas.

Las siguientes dos líneas de código ponen el color de relleno y el color de la línea de la boca. La última línea del código selecciona el tipo de arco, en este caso `ArcType.OPEN`. Los otros dos tipos son `ArcType.CHORD` y `ArcType.ROUND`.

Enseguida se agrega una leyenda:

```
Text leyenda = new Text(70, 240, "Cara Sonriente");
leyenda.setFill(Color.BLUE);
leyenda.setFont(Font.font("Verdana", 15));
```

Para este propósito se crea una instancia de la clase `Text`, la cual está en `javafx.scene.text`. El constructor toma tres parámetros, dos **double** y un `String`. Los dos primeros son usados para posicionar el texto, son las coordenadas del inicio del `String`, y el último guarda el valor del texto.

Para poner el color se usa el método `setFill()`, y luego se pone la fuente, con el método `setFont()`. En la sección [10.10](#) se verá como crear fuentes propias, pero en este caso se seleccionó el nombre "Verdana" y con un tamaño de fuente de 15 puntos.

Cuando ya se han definido todos los elementos se quiere mantenerlos juntos como un grupo. Esto se hace con la clase `Group` del paquete `javafx.scene`. Esta clase actúa como un contenedor invisible, lo cual es útil cuando ya se han definido las posiciones de las figuras, así no se deberá preocupar más acerca de cómo serán puestas dentro del contenedor:

```
Group raiz = new Group(
    cara, ojoDerecho, ojoIzquierdo, boca, leyenda);
```

Se está usando la convención de nombrar al primer nodo que se agrega a la escena como `raiz`. En este caso es el único nodo. Se crea la escena nueva de esta forma:

```
Scene escena = new Scene(raiz, 250, 275, Color.YELLOW);
```

Se escogió usar el constructor que permite poner el tamaño, ancho y alto, de la escena inicial, junto con el color de fondo. Si no se ponen estos valores inicialmente, la clase Scene, al igual que otros componentes, tienen varios métodos set que se pueden codificar después.

Ahora todo lo que falta para completar el método start() es agregar la escena al escenario, poner el título, y finalmente hacer la escena visible, lo cual se hace llamando al método show():

```
stage.setScene(escena);  
stage.setTitle("Cara Sonriente");  
stage.show();
```

Como se mencionó previamente, se ha incluido un método main() para que se pueda ejecutar la aplicación en cualquier ambiente.

```
public static void main(String[] args) {  
    launch(args);  
}
```

10.5 Manejo de Eventos en JavaFX

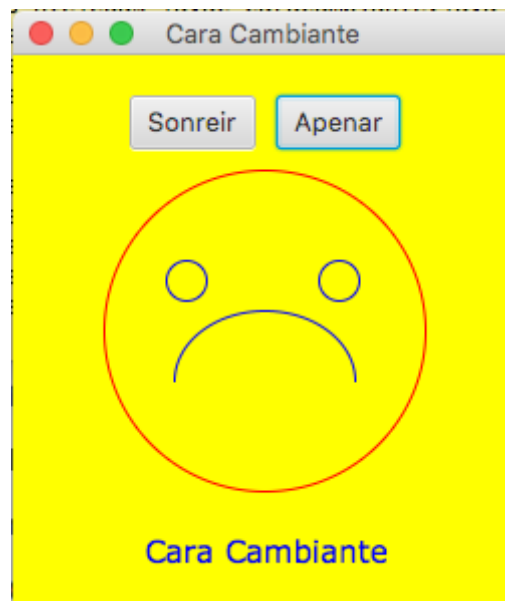
La clase CaraSonriente desarrollada en la sección previa es "pasiva" y no involucra ninguna interacción con el usuario. Las aplicaciones gráficas normalmente requerirán entrada del usuario ingresando texto o pulsando un botón.

Los controles tales como los botones necesitan responder cuando el usuario realiza alguna acción como pulsar con el ratón. En respuesta al usuario que realiza una acción, el control genera un objeto Event. Este objeto es mandado a un EventHandler el cual se une a un control particular y este proporciona las instrucciones que se harán cuando la acción ocurra. Hay muchas acciones que el usuario podría realizar, tal como presionar una tecla, arrastrar un ratón, etc., pero en este capítulo sólo se considera una pulsación simple del ratón sobre un botón.

La primera aplicación modificará la clase CaraSonriente para convertirla en la clase CaraCambiante, la cual cambia su humor para indicar tristeza o felicidad. Se agregaron un par de botones como se muestra en la figura siguiente:



Se puede observar que se cambió el título y la leyenda porque se podrá pulsar el botón *Apenar* para obtener una cara triste como se muestra enseguida. Pulsando el botón *Sonreir* se obtiene la figura de la imagen previa.



El código para la clase se muestra enseguida. Hay unos cuantos conceptos y técnicas nuevas, que son comentadas a continuación.

```
import javafx.application.Application;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.Background;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
```

```

import javafx.scene.paint.Color;
import javafx.scene.shape.Arc;
import javafx.scene.shape.ArcType;
import javafx.scene.shape.Circle;
import javafx.scene.text.Font;
import javafx.scene.text.Text;
import javafx.stage.Stage;
import javafx.geometry.Pos;

public class CaraCambiante extends Application
{
    @Override
    public void start(Stage stage) {
        // crear y configurar el círculo principal (cara)
        Circle cara = new Circle(125, 125, 80);
        cara.setFill(Color.YELLOW);
        cara.setStroke(Color.RED);

        // crear y configurar el círculo para ojo derecho
        Circle ojoDerecho = new Circle(86, 100, 10);
        ojoDerecho.setFill(Color.YELLOW);
        ojoDerecho.setStroke(Color.BLUE);

        // crear y configurar el círculo para ojo izquierdo
        Circle ojoIzquierdo = new Circle(162, 100, 10);
        ojoIzquierdo.setFill(Color.YELLOW);
        ojoIzquierdo.setStroke(Color.BLUE);

        // crear y configurar una boca sonriente
        Arc boca = new Arc(125, 150, 45, 35, 0, -180);
        boca.setFill(Color.YELLOW);
        boca.setStroke(Color.BLUE);
        boca.setType(ArcType.OPEN);

        // crear y configurar el texto
        Text leyenda = new Text(65, 240, "Cara Cambiante");
        leyenda.setFill(Color.BLUE);
        leyenda.setFont(Font.font("Verdana", 15));

        // crear un grupo que tenga todos los componentes
        Group grupo = new Group(
            cara, ojoDerecho, ojoIzquierdo, boca, leyenda);

        // crear un botón que hará la cara sonriente
        Button botonSonreir = new Button("Sonreir");

        // crear un botón que hará la cara apenada
        Button botonApenar = new Button("Apenar");
    }
}

```

```

// crear y configurar contenedor horizontal para botones
HBox cajaBotones = new HBox(10);
cajaBotones.setAlignment(Pos.CENTER);

// agregar los botones al contenedor horizontal
cajaBotones.getChildren().addAll(
    botonSonreir, botonApenar);

// crear y configurar un contenedor vertical para guardar
// la caja de botones y el grupo de la cara
VBox raiz = new VBox(10);
raiz.setBackground(Background.EMPTY);
raiz.setAlignment(Pos.CENTER);

// agregar la caja de botones y el grupo de la cara
raiz.getChildren().addAll(cajaBotones, grupo);

// crear y configurar una escena nueva
Scene escena = new Scene(raiz, 250, 275, Color.YELLOW);

// proporcionar el código cuando se presiona botón sonreir
botonSonreir.setOnAction(e -> boca.setLength(-180));

// proporcionar el código cuando se presiona botón apenas
botonApenar.setOnAction(e -> boca.setLength(180));

// agregar la escena al escenario y poner título
stage.setScene(escena);
stage.setTitle("Cara Cambiante");

// mostrar el escenario
stage.show();
}

public static void main(String[] args) {
    launch(args);
}
}

```

Se ha procedido como antes para la creación de la cara. Una vez hecho esto se han creado dos instancias de la clase `Button`. Un botón es un elemento común de la programación gráfica, y la clase `Button`, al igual que otros componentes similares se encuentran en `javafx.scene.control`. El código para los botones es:

```

Button botonSonreir = new Button("Sonreir");
Button botonApenar = new Button("Apenar");

```

Se ha usado un constructor que permite poner el texto que aparece en el botón. También se puede usar el método `setText()` de `Button` para el mismo propósito.

Teniendo creados los dos botones, se crea un contenedor para guardarlos:

```
HBox cajaBotones = new HBox(10);
cajaBotones.setAlignment(Pos.CENTER);
```

Se ha creado una instancia de un `HBox`. Este contenedor arregla los nodos contenidos horizontalmente. El constructor que ha sido usado toma un parámetro que pone la distancia entre los elementos, en este caso diez píxeles. Enseguida se ha usado el método `setAlignment()`, al cual se le manda una constante predefinida, un atributo de la clase `Pos`, la cual se encuentra en el paquete `javafx.geometry`. La constante escogida, `Pos.CENTER`, centra los componentes que el `HBox` contiene. Hay otras opciones las cuales son revisadas en la sección [10.9.1](#)

Con el `HBox` creado se agregan los botones. Se hace esto llamando al método `getChildren()`, de `HBox`, el cual regresa una lista de todos los nodos hijos. La lista tiene dos métodos para agregar los nodos, el método `add()` para incorporar un solo elemento, y `addAll()` para una lista de elementos. Se usa este último para agregar los dos botones:

```
cajaBotones.getChildren().addAll(botonSonreir, botonApenar);
```

Cuando el `HBox` contiene los botones y un `Group` contiene las figuras de la cara, estos se organizan para que la cara sea colocada verticalmente debajo de los botones, por lo que se usa un `VBox`, el cual los alinea verticalmente. Se ha dado el nombre de `raiz` a esta instancia de `VBox`, porque este será el nodo raíz que será agregado a la escena.

```
VBox raiz = new VBox(10);
raiz.setBackground(Background.EMPTY);
raiz.setAlignment(Pos.CENTER);
```

Se observa que se agrega un fondo vacío al `VBox`, esto es para que el color amarillo de la escena no quede oculto.

Ahora se puede agregar el `HBox` conteniendo los botones y el grupo conteniendo la cara, al `VBox`. Luego se agrega el `VBox` a la escena gráfica:

```
raiz.getChildren().addAll(cajaBotones, grupo);
Scene escena = new Scene(raiz, 250, 275, Color.YELLOW);
```

Enseguida se habilitan los botones para responder cuando sean presionados, y se proporciona el código que le indica a los botones que hacer cuando eso suceda. Se comentó que un control puede ser programado para generar un objeto Event en respuesta a alguna acción del usuario. El tipo de evento que será atendido es `ActionEvent`, este maneja una pulsación simple. Se necesita agregar un `EventHandler` para cada botón, para procesar el `ActionEvent` tan pronto como se pulse el ratón. Se está programando el botón para "escuchar" por una pulsación del ratón.

`EventHandler` tiene el método `handle()`, y a este método se necesita proporcionar el código para que el botón sepa qué hacer cuando se pulsa el ratón.

El código para hacer esto para cada botón se muestra enseguida y después se explica el uso de una sintaxis nueva:

```
botonSonreir.setOnAction(e -> boca.setLength(-180));  
botonApenar.setOnAction(e -> boca.setLength(180));
```

Se observa que un `Button` tiene un método llamado `setOnAction()`. Este es un ejemplo de lo que se conoce como un **método de conveniencia**, una característica de JavaFX. Es de conveniencia porque todo lo que se tiene que hacer para agregar un `EventHandler` es proporcionar el código que se necesita para su método `handle()`. La mayoría de los controles tienen estos métodos de conveniencia, todos comenzando con `setOn`. Otros ejemplos son `setOnMouseMoved()` y `setOnKeyTyped()`, entre otros.

Se puede ver que el código es mandado directamente al método, el cual se ve raro por la notación `->`. Esto se puede hacer como resultado de las innovaciones en Java 8. El código es llamado una **expresión lambda**. Las expresiones lambda permiten simplemente mandar algún código a un método como argumento, de igual forma como se hace con un tipo primitivo, o un objeto.

Las expresiones lambda serán revisadas en el segundo semestre. Por el momento esto es lo que se necesita saber para codificar el método `setOnAction()`.

En cada caso se está usando el método `setLength()` de `Arc` para redibujar la boca. Cuando se da un valor negativo se dibuja en sentido de las manecillas, entonces la boca sonríe, y dando un valor positivo hace que la cara entristezca.

Las expresiones lambda permiten proporcionar código pasándolo como un argumento al método `setOnAction()`. Las expresiones lambda usadas son:

```
e -> boca.setLength(-180)  
e -> boca.setLength(180)
```

Hay dos partes en una expresión lambda, una a cada lado del símbolo `->`. El código va a la derecha del símbolo. En la izquierda se dan nombres a los parámetros que el método, en este caso `handle()`, espera recibir un `ActionEvent`, y al cual se le ha dado el identificador `e`. Sin embargo, en este caso no será usada esta variable, pero se tiene que dar un nombre.

En el caso de que haya más de una instrucción en el código, estas se tienen que encerrar entre llaves, Por ejemplo, si se quiere pintar la boca de morado cuando sonría, la expresión lambda sería esta:

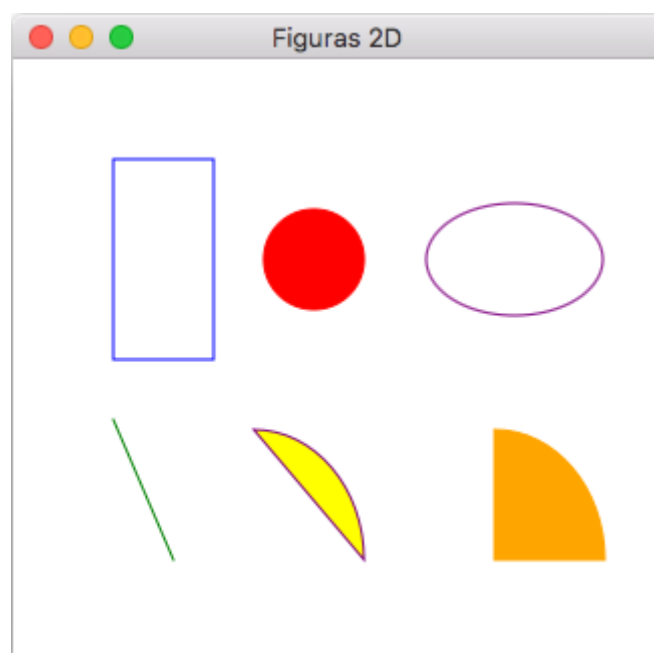
```
botonSonreir.setOnAction(e -> {  
    boca.setLength(-180);  
    boca.setStroke(Color.Violet);  
});
```

Finalmente se puede agregar la escena al escenario, poner el título y hacer visible el escenario.

```
stage.setScene(escena);  
stage.setTitle("Cara Cambiante");  
stage.show();
```

10.6 Otras formas 2D

Antes de dejar las figuras 2D, se comentan otras figuras para ampliar el repertorio, de las cuales se muestran unas cuantas en la siguiente figura:



Se puede ver que se ha experimentado con colores diferentes, y usando los métodos `setFill()` y `setStroke()`.

El rectángulo que se ve en la esquina superior izquierda fue creado con el siguiente código:

```
Rectangle rectangulo = new Rectangle(50, 50, 50, 100);
```

En el constructor usado los primeros dos parámetros, siendo todos de tipo **double**, representan las coordenadas *x* e *y* usando como origen la esquina superior izquierda, y los siguientes dos parámetros representan la base y la altura del rectángulo.

La línea que está debajo del rectángulo fue usando lo siguiente:

```
Line linea = new Line(50, 180, 80, 250);
```

Los primeros dos parámetros, en el constructor, son las coordenadas *x*, *y* de la posición inicial, y los dos últimos son las coordenadas de la posición final.

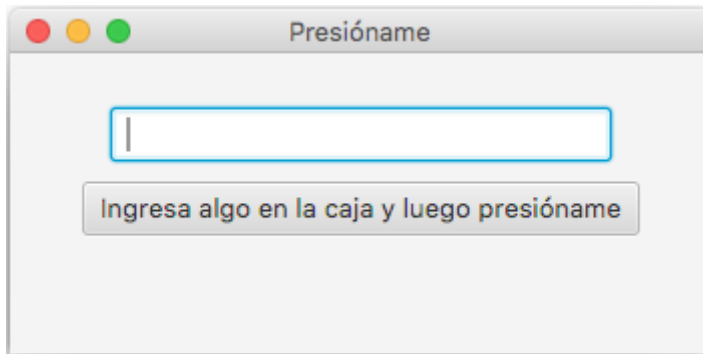
Para dibujar una elipse se puede usar `Arc` y dibujar la línea usando un ángulo de 360°. El arco puede estar cerrado usando `ArcType.CHORD` o sin línea usando `ArcType.ROUND`. Otra posibilidad es usar la clase `Ellipsis`.

Todas las figuras comentadas están en `javafx.scene.shape`. Se puede revisar otros constructores y métodos de estas y otras figuras en el sitio de Oracle.

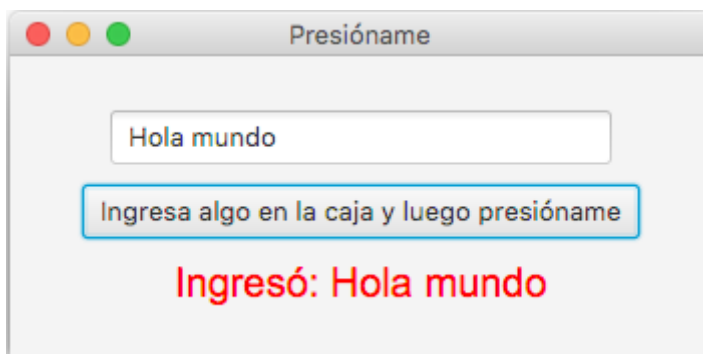
10.7 Una Clase Gráfica Interactiva

La mayoría de las aplicaciones involucra controles, como botones, casillas de verificación, campos de texto, etc., en vez de formas gráficas. La clase `Presioname` tiene controles que permiten al usuario ingresar información a través de una pantalla gráfica. El programa introduce los elementos básicos que se requieren para construir clases gráficas interactivas.

La aplicación permite al usuario ingresar texto y luego, pulsando sobre un botón, ver el texto que fue ingresado debajo del botón. Enseguida se muestra cómo se ve.



Antes de presionar el botón



Después de presionar el botón

Se muestra a continuación el código y luego se hacen comentarios.

```
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.stage.Stage;

public class Presioname extends Application {
    @Override
    public void start(Stage stage) {
        // crear y configurar un campo de texto para entrada
        TextField presionaTextField = new TextField();
        presionaTextField.setMaxWidth(250);

        // crear y configurar una etiqueta para mostrar la salida
        Label presionaLabel= new Label();
        presionaLabel.setTextFill(Color.RED);
        presionaLabel.setFont(Font.font("Arial", 20));

        // crear y configurar un botón para cambiar etiqueta
        Button presionaButton = new Button();
```

```

presionaButton.setText(
    "Ingresa algo en la caja y luego presióname");
presionaButton.setOnAction(
    e -> presionaLabel.setText(
        "Ingresó: " + presionaTextField.getText()));

// crear y configurar un VBox para guardar los componentes
VBox raiz = new VBox();
raiz.setSpacing(10);
raiz.setAlignment(Pos.CENTER);

// agregar los componentes al VBox
raiz.getChildren().addAll(
    presionaTextField, presionaButton, presionaLabel);

// crear una escena nueva
Scene scene = new Scene(raiz, 350, 150);

// agregar escena; configurar y hacerlo visible escenario
stage.setScene(scene);
stage.setTitle("Presióname");
stage.show();
}

public static void main(String[] args) {
    launch(args);
}
}

```

El cuadro en el cual se ingresa el texto es llamado `TextField`. Esto permite teclear en una línea de texto:

```

TextField presionaTextField = new TextField();
presionaTextField.setMaxWidth(250);

```

Se ha puesto el ancho máximo del `TextField` en 250, en caso de no hacerlo, este solo llenaría el ancho de su contenedor padre. Una clase similar es `TextArea`, la cual permite agregar varios renglones de texto, se revisa un ejemplo de esto en la sección siguiente.

Cuando el botón es presionado, el texto ingresado será mostrado debajo del botón en una `Label`. La etiqueta muestra texto, y se crea y configura con las siguientes líneas de código:

```

Label presionaLabel= new Label();
presionaLabel.setTextFill(Color.RED);
presionaLabel.setFont(Font.font("Arial", 20));

```

Enseguida el código para el Button:

```
Button presionaButton = new Button();
presionaButton.setText(
    "Ingresa algo en la caja y luego presióname");
presionaButton.setOnAction(
    e -> presionaLabel.setText(
        "Ingresó: " + presionaTextField.getText()));
```

Ya se revisó cómo crear y codificar un botón. La expresión lambda usa el método `setText()` de `Label` para mostrar el mensaje. El parámetro en `setText()` concatena una cadena literal introductoria con la cadena que devuelve el método `getText()` de `TextField` para obtener el texto ingresado.

Luego se crea y configura un `VBox`, se agregan tres componentes, y luego se agrega el `VBox` a la escena.

```
VBox raiz = new VBox();
raiz.setSpacing(10);
raiz.setAlignment(Pos.CENTER);

raiz.getChildren().addAll(
    presionaTextField, presionaButton, presionaLabel);

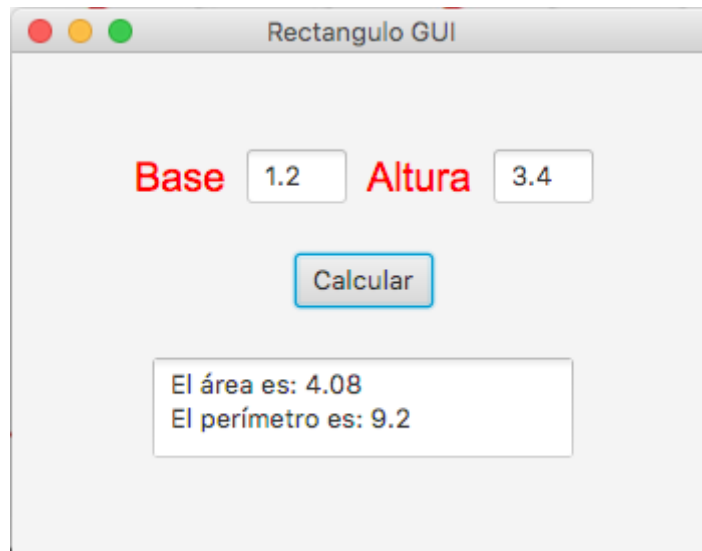
Scene scene = new Scene(raiz, 350, 150);
```

Por último se agrega la escena al escenario, se agrega el título y se hace visible.

```
stage.setScene(scene);
stage.setTitle("Presióname");
stage.show();
```

10.8 GUI para la clase Rectángulo

Los programas escritos que utilizan las clases desarrolladas están basadas en texto. Sin embargo en muchos casos se quiere crear una interfaz gráfica de usuario o GUI (*Graphical User Interface*) para las clases. Se hará esto para la clase `Rectángulo` que fue desarrollada en el capítulo [8](#). El tipo de interfaz de la que se habla se muestra enseguida:



El código para la GUI de la imagen previa es:

```
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.TextArea;
import javafx.scene.control.TextField;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.stage.Stage;

public class RectanguloGUI extends Application {
    // crear un objeto de la clase Rectangulo como un atributo
    private Rectangulo probarRect = new Rectangulo(0, 0);

    @Override
    public void start(Stage stage) {
        // crear y configurar campos de texto para entrada
        TextField campoBase = new TextField();
        campoBase.setMaxWidth(50);

        TextField campoAltura = new TextField();
        campoAltura.setMaxWidth(50);

        // crear y configurar área de texto, no editable,
        // para mostrar resultados.
        TextArea pantalla = new TextArea();
        pantalla.setEditable(false);
```

```

pantalla.setMinSize(210,50);
pantalla.setMaxSize(210,50);

// crear y configurar Labels para los campos de texto
Label etiquetaBase = new Label("Base");
etiquetaBase.setTextFill(Color.RED);
etiquetaBase.setFont(Font.font("Arial", 20));

Label etiquetaAltura= new Label("Altura");
etiquetaAltura.setTextFill(Color.RED);
etiquetaAltura.setFont(Font.font("Arial", 20));

// crear y configurar Button para realizar cálculos
Button botonCalcular = new Button();
botonCalcular.setText("Calcular");
botonCalcular.setOnAction( e -> {
    // revisar que los campos no estén vacíos
    if(campoBase.getText().isEmpty() ||
        campoAltura.getText().isEmpty()) {
        pantalla.setText("Se debe ingresar base y altura");
    }
    else {
        // convertir entradas a double y poner
        // poner base y altura Rectangulo
        probarRect.setBase(
            Double.parseDouble(campoBase.getText()));
        probarRect.setAltura(
            Double.parseDouble(campoAltura.getText()));

        // usar métodos de Rectangulo para área y perímetro
        pantalla.setText(
            "El área es: " + probarRect.calcularArea()
            + "\n" + "El perímetro es: "
            + probarRect.calcularPerimetro());
    }
}

); // cierre método setOnAction()

// crear y configurar un HBox para etiquetas y entradas
HBox componentesEntrada = new HBox(10);
componentesEntrada.setAlignment(Pos.CENTER);
componentesEntrada.getChildren().addAll(etiquetaBase,
    campoBase, etiquetaAltura, campoAltura);

// crear y configurar contenedor vertical para componentes
VBox raiz = new VBox(25);
raiz.setAlignment(Pos.CENTER);
raiz.getChildren().addAll(componentesEntrada, botonCalcular,

```

```

        pantalla);

        // crear una escena nueva y agregarla al escenario
        Scene escena = new Scene(raiz, 350, 250);
        stage.setScene(escena);
        stage.setTitle("Rectangulo GUI");
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

Para conectar una GUI a una clase, se crea un objeto de esa clase dentro de la clase GUI. Se ha declarado un atributo, probarRect, el cual se ha inicializado como un Rectangulo nuevo con una base y altura de cero, porque el usuario no ha ingresado nada todavía:

```
private Rectangulo probarRect = new Rectangulo(0, 0);
```

Después de esto se declaran los componentes gráficos; el único de estos que no se ha visto es el TextArea, el cual es una área de texto más grande como se ve en la figura previa, donde el área y el perímetro del rectángulo serán mostrado. Es un componente útil para ingresar y mostrar texto, sin embargo en esta ocasión se está usando solo para mostrar texto, no se permite entrada. Se ha declarado y configurado como sigue:

```

TextArea pantalla = new TextArea();
pantalla.setEditable(false);
pantalla.setMinSize(210,50);
pantalla.setMaxSize(210,50);

```

Para evitar que se ingrese texto se usa el método setEditable() pasándole false, y se le ha dado un tamaño fijo llamando a los métodos setMinSize() y setMaxSize().

Se debe considerar la expresión lambda que se ha enviado en el método setOnAction() del botón calcular.

```

botonCalcular.setOnAction( e -> {
    // revisar que los campos no estén vacíos
    if(campoBase.getText().isEmpty() ||
        campoAltura.getText().isEmpty()) {
        pantalla.setText("Se debe ingresar base y altura");
    }
    else {

```

```

// convertir entradas a double y poner
// poner base y altura Rectangulo
probarRect.setBase(
    Double.parseDouble(campoBase.getText()));
probarRect.setAltura(
    Double.parseDouble(campoAltura.getText()));

// usar métodos de Rectangulo para área y perímetro
pantalla.setText(
    "El área es: " + probarRect.calcularArea()
    + "\n" + "El perímetro es: "
    + probarRect.calcularPerimetro());
}
}
);

```

Lo primero que se hace es revisar que algo haya sido ingresado. Esto se hace leyendo el `String` que está actualmente en el `TextField` llamando al método `getText()`, y luego llamando al método `isEmpty()` de `String`.

Si alguno de los campos está vacío entonces un mensaje de error es mostrado; de otra forma se continúa con la tarea. Si se hubiera querido se podría hacer más validación de la entrada, por ejemplo se podría haber revisado que cero o valores negativos no sean ingresados. O, si se quisiera ser más estricto acerca de la definición de "rectángulo", se podría haber revisado que los dos lados adyacentes no sean iguales.

Si no hay error, se usan los métodos `setBase()` y `setAltura()` de `Rectangulo` para poner la base y la altura de `probarRect` a los valores ingresados. Sin embargo, estos métodos esperan recibir **double**, pero el método `getText()` de `TextField`, el cual se usa para ver que ha sido ingresado, regresa un `String`.

Se debe hacer por lo tanto una conversión. Para hacer esto se usa el método `parseDouble()` de la clase `Double`, una clase envoltura que se revisó en el capítulo [8](#). El método `parseDouble()` toma un `String` y la convierte a un **double**:

```

probarRect.setBase(Double.parseDouble(campoBase.getText()));
probarRect.setAltura(Double.parseDouble(campoAltura.getText()));

```

Podría ocurrir que si el `String` no contiene un número, entonces esto podría causar un problema. En el segundo semestre se verá cómo manejar este tipo de error.

Se debe comentar que si se quiere convertir un `String` a **int**, se tiene que usar el método `parseInt()` de la clase `Integer`.

Si se quiere hacer lo contrario convertir un **double** o un **int** a un **String** se puede simplemente concatenar con una cadena vacía, como se muestra a continuación:

```
String sEnt = "" + 3;  
String sReal = "" + 3.14;
```

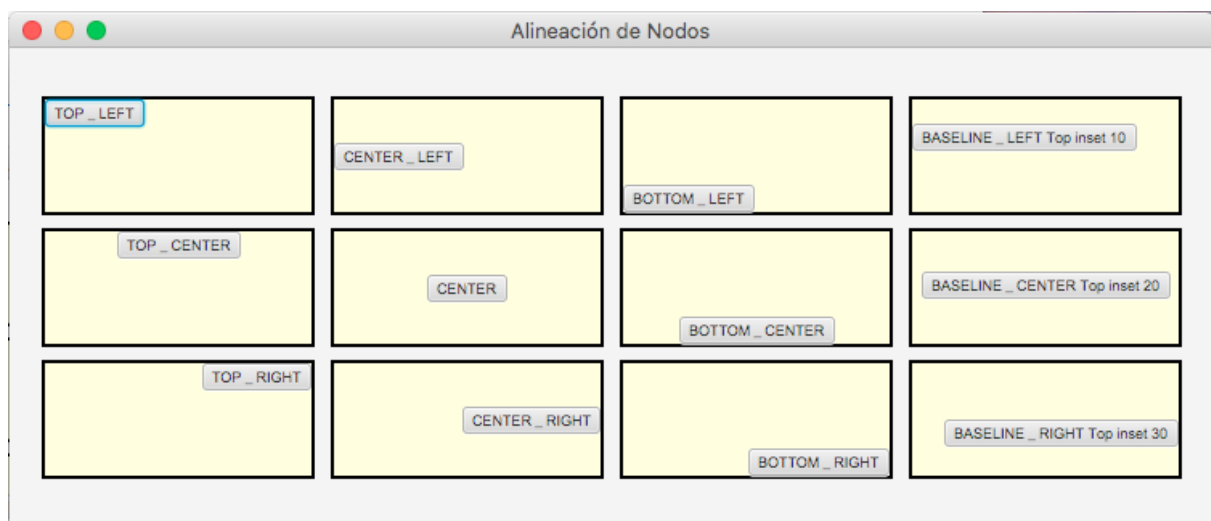
El resto del código arregla los elementos usando un **HBox** para guardar las etiquetas y campos de entrada para que estén alineados horizontalmente, y luego se usa un **VBox** para alinearlos verticalmente con el botón y el área de pantalla. Esto es algo que deberá ser usado a partir de ahora. En la siguiente sección se explica más como usar otros contenedores.

10.9 Contenedores y Diseños

Con los contenedores **HBox** y **VBox** se pueden lograr varias cosas gracias a su versatilidad, y para aplicaciones simples se puede hacer mucho con estos dos contenedores. Se inicia esta sección comentando un poco más de lo que se puede hacer con estos, y luego se muestran otros contenedores con políticas de diseño diferente.

10.9.1 Otras funciones de HBox y VBox

En la siguiente figura se pueden ver doce contenedores **VBox**, aunque también podrían ser **HBox** ya que cada uno contiene un solo componente, organizados en cuatro grupos de tres. Se ha dibujado un borde alrededor y se ha coloreado el fondo. Cada caja contiene un **Button**, la presencia del borde muestra el efecto de configurar la alineación a valores diferentes.



Hasta ahora el único alineamiento visto fue `Pos.CENTER`, pero hay otros once que pueden ser seleccionados. La mayoría son autoexplicativos, excepto los tres de la derecha que requieren un poco de explicación. `Pos.BASELINE_LEFT`, `Pos.BASELINE_CENTER` y `Pos.BASELINE_RIGHT` colocan el componente en la posición más baja disponible, y son más relevantes a campos de texto donde se quiere que aparezcan en la parte baja de una ventana, como en una aplicación de conversación. En el ejemplo se han puesto las cajas para que tengan un recuadro superior, y los botones son colocados en consecuencia.

Para poner algún recuadro en un componente se usa el método `setPadding()`, con una sentencia como esta:

```
box.setPadding(new Insets(10, 20, 10, 20));
```

Los parámetros del constructor `Insets` son todos **double**, y definen el recuadro superior, derecho, inferior e izquierdo respectivamente. Un sólo parámetro pondrá los cuatro recuadros con el mismo valor.

`box` podría ser cualquier componente tal como un `VBox` o `HBox`, o un `Button`, `Label` o `TextField` por ejemplo, ya que todos heredan el método `setPadding()` de una clase superior.

Para crear un borde negro se hizo lo siguiente:

```
box.setBorder(new Border(new BorderStroke(Color.BLACK,  
    BorderStrokeStyle.SOLID, new CornerRadii(0), new BorderWidths(2))));
```

El método `setBorder()` requiere un objeto de la clase `Border`. Para crearlo se tiene que mandar al constructor un objeto de `BorderStroke`, el cual requiere cuatro argumentos: color, estilo, redondeo de esquina y ancho del borde. `CornerRadii` determina la redondez de las esquinas, con un valor de cero se producen esquinas cuadradas.

Para lograr el color de fondo se hizo esto:

```
box.setBackground(new Background(new BackgroundFill(  
    Color.LIGHTYELLOW, new CornerRadii(0), new Insets(0))));
```

Con lo revisado previamente se puede entender lo que sucede.

Todos estos fondos y bordes están en `javafx.scene.layout`.

En la sección [10.10](#) se revisa más los bordes y los colores.

10.9.2 GridPane

Un GridPane es un contenedor muy útil. Este coloca los componentes en una matriz de renglones y columnas como se muestra en la siguiente figura:



Las siguientes líneas de código crean un objeto GridPane, lo configura para posicionar los componentes en el centro de cada celda, y dejar un espacio vertical de 10 píxeles (el espacio entre renglones) y de 5 píxeles horizontalmente (el espacio entre columnas).

```
GridPane pane = new GridPane();
pane.setAlignment(Pos.CENTER);
pane.setVgap(10);
pane.setHgap(5);
```

GridPane es dimensionado dinámicamente conforme se insertan los componentes, como son las celdas individuales. Por ejemplo, se podría insertar un Button, miBoton, al objeto GridPane anterior como sigue:

```
pane.add(miBoton, indiceColumna, indiceRenglon);
```

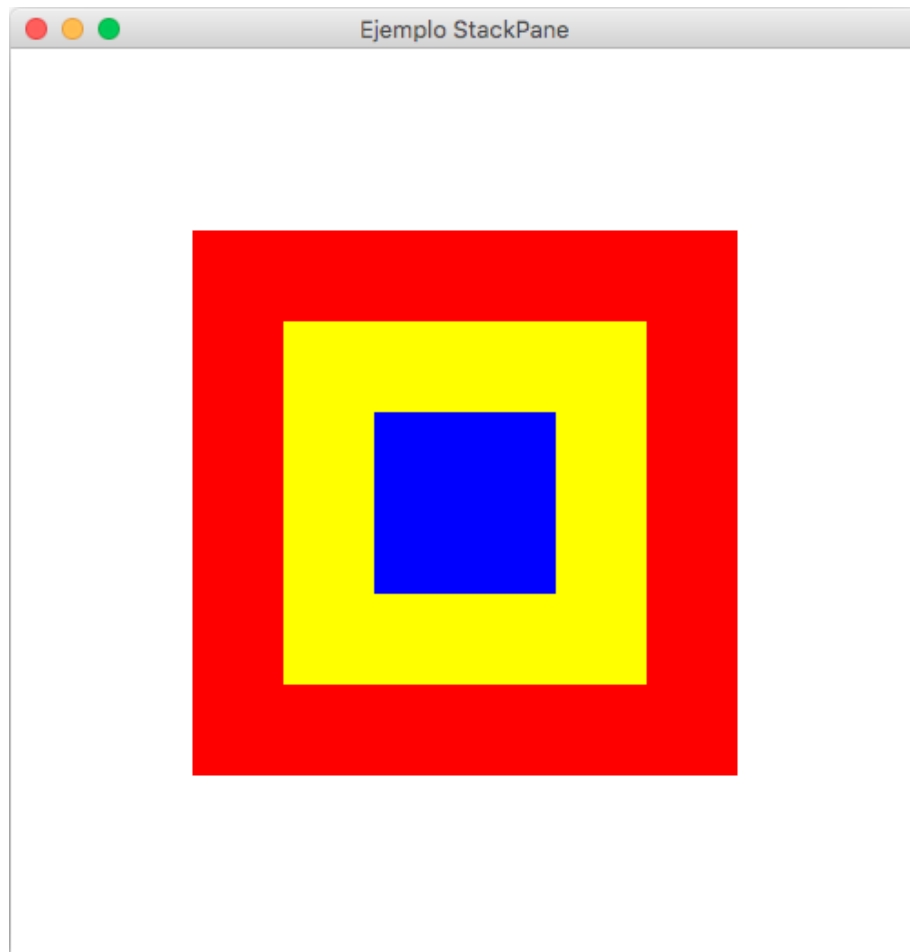
indiceColumna e indiceRenglon son `int`, y se inicia contando desde cero, así la siguiente línea de código agrega el botón en la columna 4, renglón 6:

```
pane.add(miBoton, 3, 5);
```

GridPane debido a su flexibilidad puede ser muy versátil y permite crear presentaciones complejas.

10.9.3 StackPane

Un StackPane apila componentes en la cima de otros. En la siguiente figura se han creado tres cuadros diferentes coloreados, cada uno menor que el previo, y agregados al StackPane a partir del más grande y hasta el más pequeño.



Los componentes son agregados llamando al método `getChildren()` heredado por `StackPane`. Para el ejemplo se escogió alinearlos en el centro del panel.

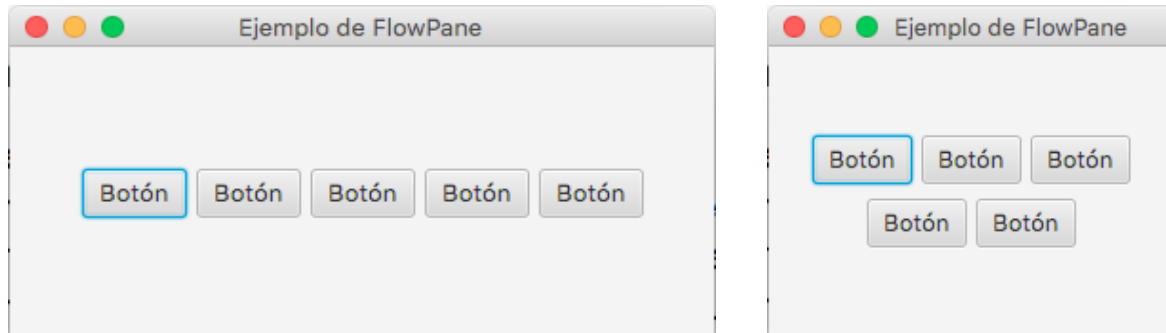
Hay otra forma útil en la que se puede usar este contenedor, creando varios elementos cada uno del mismo tamaño y apilándolos uno encima del otro. Luego se puede escoger cual será visible, así la pila se comporta como un paquete de cartas. De esta forma se puede crear una serie de pantallas las cuales podrían, por ejemplo, ser las formas que requieren ser llenadas progresivamente.

10.9.3 FlowPane y BorderPane

Previo a la existencia de JavaFX el paquete principal para producir gráficos en Java fue *Swing*. Con *Swing*, la forma como un contenedor acomoda sus componentes (su *política de diseño*) estaba determinada por la asociación a un *manejador de diseño* particular con este. Dos de los manejadores más comunes fueron `FlowLayout` y `BorderLayout`. JavaFX trabaja proporcionando contenedores diferentes, cada uno con su propia política de diseño, y con la existencia de `VBox` y `HBox`, las políticas de diseño de flujo y borde no son tan útiles. Sin embargo, dos contenedores que

producen resultados similares a estos se proporcionan en JavaFX, FlowPane y BorderPane.

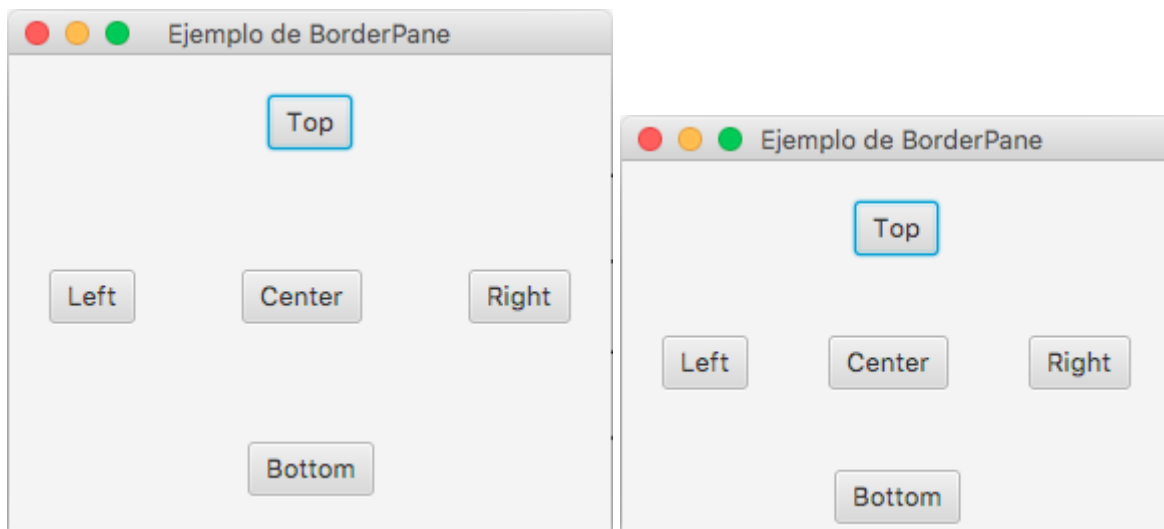
Un FlowPane opera arreglando los elementos en un renglón, en el orden en el que fueron agregados, iniciando un renglón nuevo cuando sea necesario. Si la ventana es redimensionada, los componentes se acomodan en consecuencia como se muestra enseguida en las ventanas redimensionadas.



Al igual que con VBox y HBox, los nodos son agregados a un objeto FlowPane llamando al método `getChildren()` y luego usando `add()` o `addAll()`.

El BorderPane opera dividiendo la ventana en cinco regiones llamadas *Top*, *Bottom*, *Left*, *Right* y *Center*.

Si se usa un BorderPane los componentes no se mueven alrededor cuando la ventana es redimensionada, como se puede ver en la siguiente figura.



Para agregar un elemento llamado `miBoton`, por ejemplo, a la región *top* de un BorderPane llamado `pane`, se podría hacer lo siguiente:

```
pane.setTop(miBoton);
```

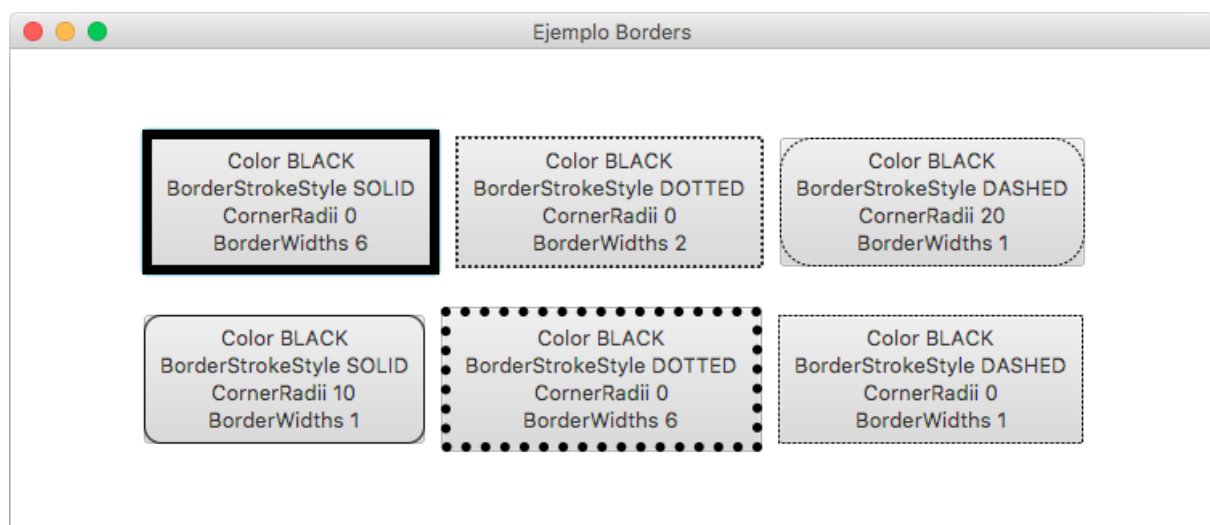
De igual forma, para las otras regiones `BorderPane` tiene los métodos `setBottom()`, `setLeft()`, `setRight()` y `setCenter()`.

10.10 Bordes, Fuentes y Colores

Se han visto ejemplos de como colocar bordes alrededor de componentes y como se puede determinar el color del texto y el fondo, y definir fuentes diferentes. En esta sección se darán unas cuantas formas para mejorar las aplicaciones.

10.10.1 Bordes

En la siguiente figura se observan seis botones con diferentes estilos de bordes.



En la sección [10.9.1](#) se indicó que para poner un borde a un componente tal como un botón se usa el método `setBorder()`, este requiere un objeto de la clase `Border`, el cual es creado con un objeto de la clase `BorderStroke`.

Por ejemplo, para el primer botón, `boton1`, se crea el siguiente `BorderStroke`:

```
BorderStroke stroke1 = new BorderStroke(Color.BLACK,  
    BorderStrokeStyle.SOLID, new CornerRadii(0), new BorderWidths(6));
```

Luego se aplica al botón:

```
boton1.setBorder(new Border(stroke1));
```

Un efecto más es crear un borde con título como se muestra enseguida:



Para hacer lo anterior se usa un `TitledPane`, el cual es una clase de control, y reside en el paquete `javafx.scene.control`. El cristal viene con una flecha hacia abajo que permite colapsar el cristal, como se observa en la segunda caja. Si se quiere solo un título para propósitos decorativos se puede apagar esta característica como en la primera caja. En cada caso un `VBox` fue agregado al panel; otros nodos podrían ser agregados al `VBox`.

El código para crear el primer cristal es el siguiente:

```
VBox box1 = new VBox();
box1.setMinSize(100,75);
TitledPane primerPane = new TitledPane("Pane 1",box1);
primerPane.setCollapsible(false);
```

10.10.2 Fuentes

En la siguiente figura se muestran algunos ejemplo diferentes de fuentes:



En el ejemplo se han aplicado fuentes a varios objetos `Text`. Para lograrlo se usó el método `setFont()` de `Text`, controles tales como `Button` también tienen un método `setFont()`.

Hay dos maneras de hacer esto. Primeramente se puede usar el método `font()` de `Font`. En el primer ejemplo, suponiendo que el objeto `Text` es llamado `leyenda1`, se podría escribir:

```
Font fuente1=Font.font("Verdana",FontWeight.BOLD,  
                        FontPosture.ITALIC,12);  
leyenda1.setFont(fuente1);
```

La otra forma de hacerlo es crear una fuente nueva con uno de los dos constructores. El primero requiere solo el tamaño de la fuente (un **double**) y usa la fuente del sistema por defecto. El segundo requiere el nombre de la fuente y el tamaño. El quinto ejemplo se logra así:

```
Font fuente5=new Font("Calibri",38);
```

El subrayado es hecho usando el método `setUnderline()` del componente.

10.10.3 Colores

Se ha visto que la clase `Color` JavaFX tiene muchos colores predefinidos. Sin embargo, si se quiere se pueden crear colores propios. Para lograrlo se usan los tres colores primarios rojo, verde y azul; el resto de los colores puede ser obtenido mezclando estos en proporciones diferentes.

Mezclando rojo, verde y azul en igual intensidad produce luz blanca; el color negro es la ausencia de los tres. Mezclando cantidades iguales de rojo y verde, sin azul, produce luz amarilla; rojo y azul producen un color malva llamado magenta; y mezclando azul y verde se produce cian, un tipo de turquesa.

La clase `Color` tiene un método llamado `rgb()` el cual permite especificar la intensidad de cada color primario, rojo, verde, azul respectivamente. La intensidad para cada color varía desde cero hasta 255. Así que hay 256 intensidades posibles para cada color primario, por lo que la cantidad total de colores diferentes disponibles es 16,777,216.

Como ejemplo, se podría crear el color siguiente:

```
Color color1 = Color.rgb(200, 100, 50);
```

Luego se podría poner el color del texto de un botón, `boton1`, así:

```
boton1.setTextFill(color1);
```

Esta combinación particular da un tipo de naranja.

10.11 Formato de Números

La siguiente técnica no está específicamente relacionada a JavaFX, pero es algo que se quiere usar frecuentemente en las aplicaciones gráficas. Se requiere que la salida numérica aparezca en un formato conveniente, por ejemplo con no más de dos números después del punto decimal. Para lograrlo se puede usar la clase `DecimalFormat` que está en el paquete `java.text`. Se requerirá tener la siguiente sentencia:

```
import java.text.DecimalFormat;
```

Una vez que se tiene acceso a esta clase se pueden crear objetos `DecimalFormat` en el programa. Estos objetos pueden ser usados para formatear números. El constructor `DecimalFormat` tiene un sólo parámetro, la cadena de formato. Esta cadena indica al objeto como formatear un número decimal dado. Algunos elementos importantes de la cadena son dados enseguida:

Carácter	Significado
.	Insertar un punto decimal
,	Insertar una coma
0	Mostrar un sólo dígito
#	Mostrar un sólo dígito o vacío si no hay dígito presente

En el ejemplo de la sección siguiente se creará el siguiente objeto `DecimalFormat`:

```
DecimalFormat fd = new DecimalFormat("0.0#");
```

Aquí el objeto `DecimalFormat`, `fd`, está siendo informado sobre cómo formatear cualquier número que se le pudiera dar, como se muestra en el ejemplo de la siguiente sección.

Teniendo creado un objeto `DecimalFormat`, se podría entonces crear un `String` `s`, desde un `double`, `d`, como sigue:

```
String s = fd.format(d);
```

El programa siguiente muestra algunos ejemplos:

```
import java.text.DecimalFormat;

public class EjemploFormatoNumero {
    public static void main(String[] args) {
```

```

double num = 5647.3829;

DecimalFormat fd1 = new DecimalFormat("###,##0.0#");
DecimalFormat fd2 = new DecimalFormat("###000.00");
DecimalFormat fd3 = new DecimalFormat("00.0");
DecimalFormat fd4 = new DecimalFormat("000000.00000");
DecimalFormat fd5 = new DecimalFormat("000,000.00####");

System.out.println(fd1.format(num));
System.out.println(fd2.format(num));
System.out.println(fd3.format(num));
System.out.println(fd4.format(num));
System.out.println(fd5.format(num));
}
}

```

10.12 Un Convertidor Métrico

El ejemplo final del capítulo es una aplicación práctica que usa todo lo revisado de JavaFX. El programa convierte entre el sistema métrico y el sistema imperial.

Se construirá la clase `ConvertidorMetrico`. La figura siguiente muestra lo que se obtiene.



La aplicación tendrá un `VBox` como su raíz, este `VBox` guardará tres `HBox`, uno por cada renglón de la figura previa. Cada renglón requiere dos `Button` los cuales harán los cálculos en cualquier dirección; estos dos `Button` se guardarán juntos en un `VBox`.

Enseguida se muestra el código para la clase `ConvertidorMetrico`; parece largo, pero la mayoría de esto ya se revisó.

```
import java.text.DecimalFormat;
```

```

import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class ConvertidorMetrico extends Application {
    @Override
    public void start(Stage stage) {
        // ver sección previa
        DecimalFormat df = new DecimalFormat("0.0#");

        // componentes para convertir entre pulgadas y centímetros

        // crear campos de entrada y etiquetas de unidades
        TextField textoCm = new TextField();
        Label etiquetaCm = new Label("Cm");
        TextField textoPlg = new TextField();
        Label etiquetaPlg = new Label("Pulg ");

        // crear botones para realizar cálculos
        Button botonCmPlg = new Button(" --=> ");
        Button botonPlgCm = new Button(" <== ");

        // crear un VBox para guardar los botones
        VBox botonesPlgCm = new VBox();
        botonesPlgCm.getChildren().addAll(botonCmPlg, botonPlgCm);

        // crear un HBox para guardar los elementos del 1er renglón
        HBox panelPlgCm = new HBox(10); // contenedor compuesto
        panelPlgCm.getChildren().addAll(
            textoCm, etiquetaCm, botonesPlgCm, textoPlg, etiquetaPlg);
        panelPlgCm.setAlignment(Pos.CENTER);

        // componentes para convertir entre millas y kilómetros

        // crear campos de entrada y etiquetas de unidades
        TextField textoKm = new TextField();
        Label etiquetaKm = new Label("Km");
        TextField textoMilla = new TextField();
        Label etiquetaMilla = new Label("Milla");

        // crear botones para realizar cálculos
        Button botonKmMilla = new Button(" --=> ");
        Button botonMillaKm = new Button(" <== ");
    }
}

```

```

// crear un VBox para guardar los botones
VBox botonesMillaKm = new VBox();
botonesMillaKm.getChildren().addAll(botonKmMilla, botonMillaKm);

// crear un HBox para guardar los elementos del 2º renglón
HBox panelMillaKm = new HBox(10);
panelMillaKm.getChildren().addAll(
    textoKm, etiquetaKm, botonesMillaKm, textoMilla, etiquetaMilla);
panelMillaKm.setAlignment(Pos.CENTER);

// componentes para convertir entre libras y kilogramos

// crear campos de entrada y etiquetas de unidades
TextField textoKg = new TextField();
Label etiquetaKg = new Label("Kg ");
TextField textoLb = new TextField();
Label etiquetaLb = new Label("Lb ");

// crear botones para realizar cálculos
Button botonKgLb = new Button(" --=> ");
Button botonLbKg = new Button(" <== ");

// crear un VBox para guardar los botones
VBox botonesLbKg = new VBox();
botonesLbKg.getChildren().addAll(botonKgLb, botonLbKg);

// crear un HBox para guardar los elementos del 3er renglón
HBox panelLbKg = new HBox(10);
panelLbKg.getChildren().addAll(
    textoKg, etiquetaKg, botonesLbKg, textoLb, etiquetaLb);
panelLbKg.setAlignment(Pos.CENTER);

// crear un VBox para guardar los tres renglones
VBox raiz = new VBox(10);
raiz.getChildren().addAll(panelPlgCm, panelMillaKm, panelLbKg);
raiz.setAlignment(Pos.CENTER);

// escribir el código para los botones

botonCmPlg.setOnAction(
    e -> {
        String s = new String(textoCm.getText());
        double d = Double.parseDouble(s);
        d = d / 2.54;
        s = df.format(d);
        textoPlg.setText(s);
    }
);

botonPlgCm.setOnAction(

```

```

        e -> {
            String s = new String(textoPlg.getText());
            double d = Double.parseDouble(s);
            d = d * 2.54;
            s = df.format(d);
            textoCm.setText(s);
        }
    );

    botonKmMilla.setAction(
        e -> {
            String s = new String(textoKm.getText());
            double d = Double.parseDouble(s);
            d = d / 1.609;
            s = df.format(d);
            textoMilla.setText(s);
        }
    );

    botonMillaKm.setAction(
        e -> {
            String s = new String(textoMilla.getText());
            double d = Double.parseDouble(s);
            d = d * 1.609;
            s = df.format(d);
            textoKm.setText(s);
        }
    );

    botonKgLb.setAction(
        e -> {
            String s = new String(textoKg.getText());
            double d = Double.parseDouble(s);
            d = d * 2.2;
            s = df.format(d);
            textoLb.setText(s);
        }
    );

    botonLbKg.setAction(
        e -> {
            String s = new String(textoLb.getText());
            double d = Double.parseDouble(s);
            d = d / 2.2;
            s = df.format(d);
            textoKg.setText(s);
        }
    );

    // crear una escena nueva

```

```
Scene scene = new Scene(raiz);

// agregar la escena al escenario, configurarla y hacerla visible
stage.setScene(scene);
stage.setTitle("Convertidor Métrico");
stage.setWidth(550);
stage.setHeight(250);
stage.show();
}

public static void main(String[] args) {
    launch(args);
}
}
```

El código para cada botón es muy parecido, difiriendo solamente en el cálculo particular que se necesita hacer. Revisar como se ha usado la clase `DecimalFormat` que fue descrita en la sección previa. Se declaró un objeto de esta clase al inicio del método `start()`, y se aplicó a cada uno de los botones. El formato particular escogido siempre mostrará al menos un dígito después del punto decimal una vez que el cálculo ha sido hecho.

10.13 Preguntas

1. Describir brevemente la historia de la programación gráfica en Java.
2. ¿Cuál es el nombre de los tres métodos que son llamados cuando una aplicación JavaFX es lanzada? ¿Cuál es el propósito de cada uno?
3. Describir cómo los siguientes contenedores acomodan los nodos que contienen: a) un `VBox`; b) un `HBox`; c) un `GridPane`; d) un `StackPane`; e) un `FlowPane`; f) un `BorderPane`.